МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)
Факультет информационных технологий
Кафедра «Информатика и информационные технологии»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Базы данных»

Тема «Информационная система для парикмахерской»

Вариант 24

Направление 09.03.02 «Информационные системы и технологии»
Выполнил студент группы 251-951
Руководитель проекта канд. техн. наук, доцент

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**
1 ПОСТАНОВКА ЗАДАЧИ **4**
1.1 Описание предметной области 4
1.2 Требования к информационной системе 4
1.3 Диаграмма вариантов использования 5
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА **7**
2.1 Инфологическое моделирование 7
2.2 Реляционная модель и нормализация 8
2.3 Архитектура приложения и реализация 11
2.4 Серверная логика и безопасность данных 12
3 ОСНОВНЫЕ СЦЕНАРИИ РАБОТЫ **14**
3.1 Авторизация и разграничение прав доступа 14
3.2 Сценарии работы администратора 14
3.3 Сценарии работы мастера 16
3.4 Сценарии работы клиента 17
ЗАКЛЮЧЕНИЕ **19**
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **20**
ПРИЛОЖЕНИЕ А Листинг модуля базы данных DBManager **21**
ПРИЛОЖЕНИЕ Б Триггер и представления базы данных **22**
ПРИЛОЖЕНИЕ В Скрипты разграничения прав доступа **23**

ВВЕДЕНИЕ

Предприятия сферы услуг сталкиваются с постоянной необходимостью упорядочивания сведений о клиентах, персонале и процедурах. Ручной учет в журналах приводит к регулярным ошибкам оператора. Теряются записи о постоянных посетителях, возникают накладки в расписании мастеров. Затрудняется расчет накопительных скидок и подведение финансовых итогов сети.

Для парикмахерской с филиалами важно централизованно хранить прейскурант, историю посещений и данные о мастерах. Внедрение базы данных решает эти затруднения. Программа берет на себя строгий контроль цен, фиксацию визитов и разграничение прав доступа.

Цель работы состоит в проектировании и разработке информационной системы парикмахерской с тремя категориями пользователей.

Для достижения поставленной цели требуется решить комплекс взаимосвязанных задач. Сначала необходимо изучить предметную область и сформулировать требования к системе. Затем нужно составить концептуальную модель данных Чена. Далее требуется разработать реляционную схему и доказать ее соответствие нормальной форме Бойса-Кодда. Также необходимо создать триггер расчета скидки и аналитические представления в PostgreSQL. После этого следует спроектировать архитектуру на базе Qt по шаблону Model-View-Controller и создать графический интерфейс.

Объектом исследования выступает автоматизация учета услуг и взаимодействия с клиентами. Предметом исследования считаются структуры реляционных таблиц, алгоритмы клиент-серверного взаимодействия и интерфейсные решения программы. Практическая ценность проекта заключается в создании готового программного комплекса для реального салона.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Предметной областью проекта выступает деятельность сети парикмахерских салонов. В состав организации входят территориально распределенные отделения. Каждый филиал обладает названием, физическим адресом и контактным телефоном.

В филиалах работают сотрудники. Для каждого работника фиксируются фамилия, имя, отчество, телефон и должность. Основной должностью считается мастер-парикмахер.

Клиентами парикмахерской выступают физические лица. При регистрации посетителя указываются его фамилия, имя, отчество и уникальный номер мобильного телефона. Телефон служит главным идентификатором человека при поиске перед оформлением процедуры.

Перечень услуг салона включает виды мужских, женских и универсальных стрижек. Стрижки имеют наименования и половую принадлежность. Стоимость периодически пересматривается руководством. По этой причине необходимо сохранять историю изменений цен с указанием даты ввода новых тарифов.

Факт обслуживания фиксируется учетным документом. В документе отражаются филиал, мастер, клиент, стрижка, дата операции, ставка скидки и сумма к оплате. Программа лояльности предусматривает автоматическое начисление скидки 3% при совершении клиентом пяти и более визитов.

1.2 Требования к информационной системе

К информационной системе предъявляются функциональные и системные требования. Система обязана поддерживать многопользовательский режим с разделением прав по трем категориям учетных записей. К ним относятся администратор, мастер и клиент.

Администратор наделяется правами полного контроля над справочниками. Ему доступны добавление, редактирование и удаление филиалов, стрижек и учетных записей пользователей. Также администратор вносит новые расценки в прейскурант и просматривает сводную финансовую отчетность.

Мастер получает доступ к журналу визитов своего филиала. Мастер имеет право регистрировать новых посетителей и оформлять завершенные процедуры. При оформлении мастер указывает телефон клиента, после чего программа сама находит человека и рассчитывает скидку. Функции удаления записей мастеру недоступны.

Клиент имеет доступ к личному кабинету. В кабинете отображаются история визитов, общий счетчик полученных услуг, текущий статус скидки и суммарные расходы. Клиент может изменять свой телефон и пароль.

В части безопасности требуется обеспечить защиту от несанкционированного доступа. Пароли обязаны содержать не менее 8 знаков с включением цифр, заглавных букв и специальных символов. Все запросы к серверу баз данных должны выполняться с параметрическим связыванием для предотвращения SQL-инъекций.

1.3 Диаграмма вариантов использования

Для формализации требований составлена диаграмма вариантов использования в нотации UML. На схеме отражены действующие лица и доступные им функциональные прецеденты.

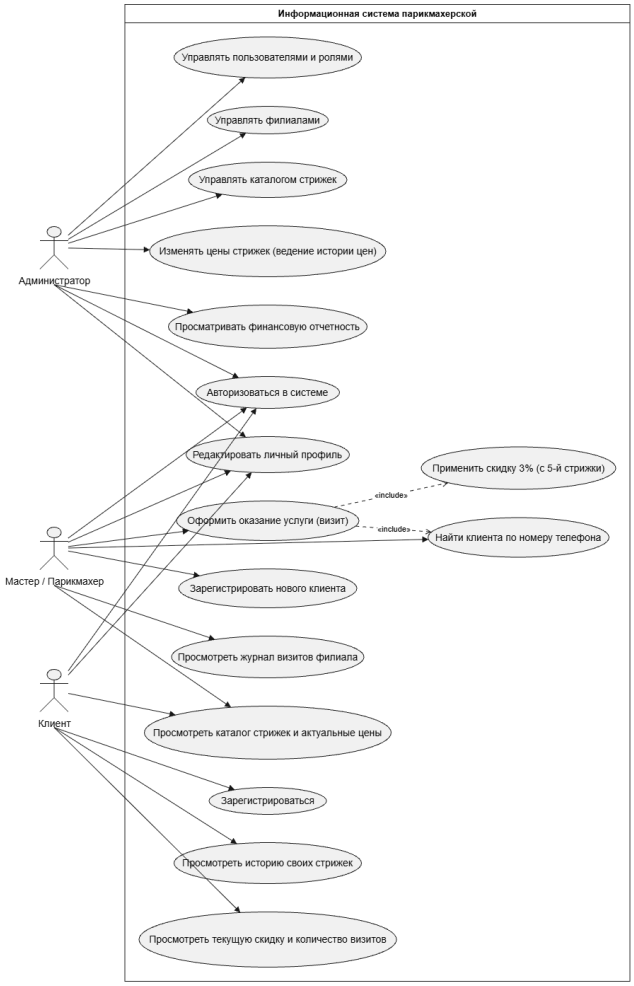


Рисунок 1 – Диаграмма вариантов использования информационной системы

Анализ представленной диаграммы показывает разделение обязанностей между участниками. Главными акторами выступают Администратор, Мастер-парикмахер и Клиент. Прецедент авторизации считается общим для всех участников. После входа рабочее пространство перестраивается под права пользователя. Администратор управляет справочниками, мастер фиксирует оказание услуг, а клиент осуществляет личный мониторинг.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА

2.1 Инфологическое моделирование

На этапе инфологического проектирования создается семантическая модель предметной области. Концептуальная схема построена с использованием классической нотации сущностей и связей Питера Чена.

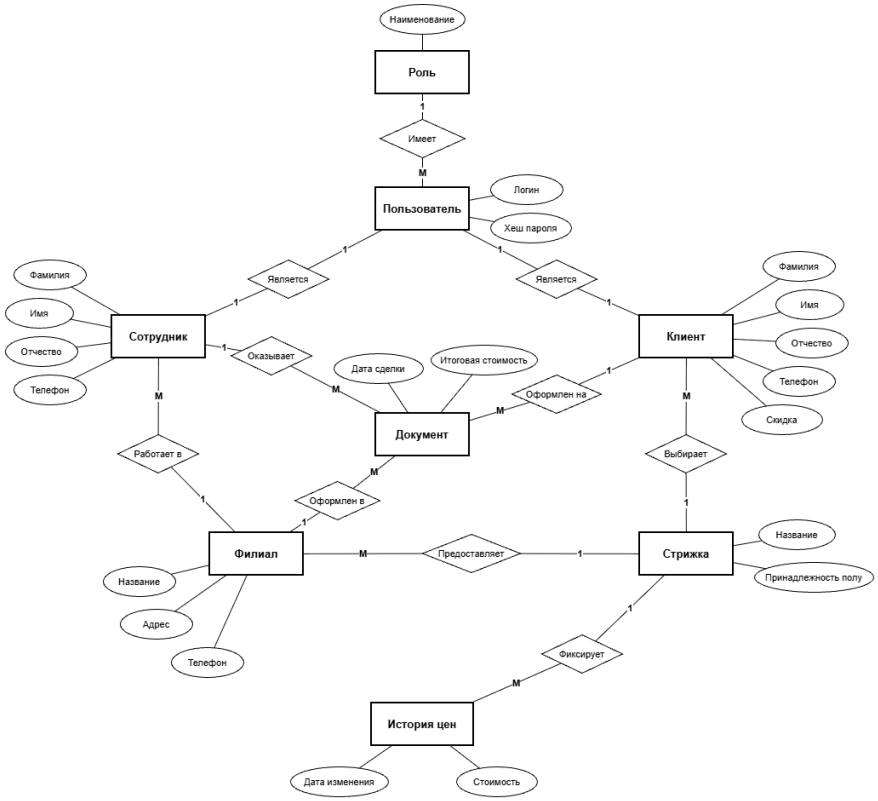


Рисунок 2 – Инфологическая модель базы данных парикмахерской

В инфологической модели выделено восемь смысловых сущностей. Каждая сущность обладает определенным набором атрибутов.

Сущность Роль хранит наименование типа доступа в системе. Связь с пользователями имеет характер один ко многим.

Сущность Пользователь содержит учетные данные для авторизации, включая логин и хеш пароля. Пользователь связан отношением один к одному с сотрудником либо с клиентом.

Сущность Сотрудник описывает персонал парикмахерской. Атрибутами выступают фамилия, имя, отчество, телефон и должность. Сотрудник работает в конкретном филиале, образуя связь многие к одному.

Сущность Филиал аккумулирует данные об отделениях салона. Она хранит название, адрес и телефон. Филиал предоставляет стрижки клиентам через своих сотрудников.

Сущность Клиент объединяет персональные данные посетителей, включая телефон и накопительную скидку. Клиент оформляет визиты и выбирает желаемые стрижки.

Сущность Стрижка содержит название прически и указание пола. Стрижка связывается с историей цен отношением один ко многим.

Сущность История цен фиксирует стоимость каждой стрижки с точной временной меткой введения тарифа.

Центральной сущностью выступает Документ. Он фиксирует факт оказания услуги, соединяя филиал, мастера, клиента и стрижку. Документ содержит дату сделки, ставку скидки и окончательную стоимость.

2.2 Реляционная модель и нормализация

На основе инфологической диаграммы разработана физическая реляционная схема базы данных. Схема состоит из девяти таблиц, связанных внешними ключами с контролем ссылочной целостности.

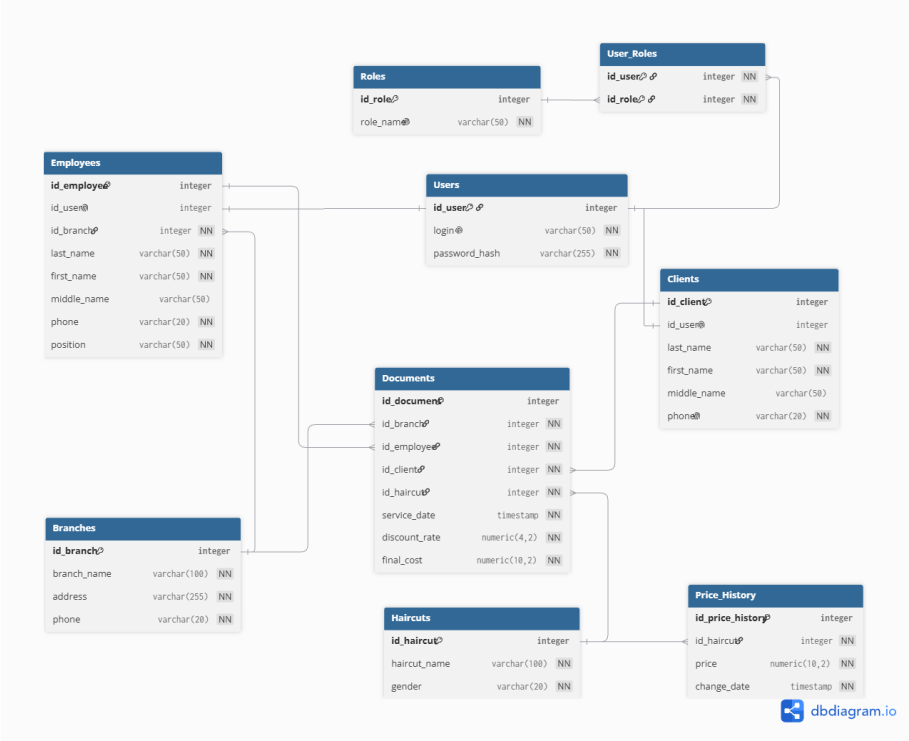


Рисунок 3 – Реляционная модель базы данных

Процесс нормализации гарантирует отсутствие дублирования информации и исключает аномалии. Отношение находится в нормальной форме Бойса-Кодда, если в каждой его нетривиальной функциональной зависимости детерминанта представляет собой потенциальный ключ или суперключ.

Отношение Roles предназначено для классификации прав доступа. Таблица содержит id_role и role_name. Первичным ключом выступает id_role, а альтернативным ключом считается role_name. Существуют зависимости id_role -> role_name и role_name -> id_role. Детерминантами выступают потенциальные ключи. Условие НФБК строго выполняется.

Отношение Users служит для хранения учетных записей. Таблица включает id_user, login и password_hash. Ключами отношения считаются id_user и login. Зависимости имеют вид id_user -> login, password_hash и login -> id_user, password_hash. Детерминантами выступают только потенциальные ключи. НФБК выполняется.

Отношение User_Roles реализует связь многие ко многим между пользователями и ролями. Таблица состоит из двух внешних ключей id_user и id_role. Составным первичным ключом выступает пара {id_user, id_role}. Нетривиальные функциональные зависимости между отдельными атрибутами отсутствуют. Отношение находится в НФБК.

Отношение Branches хранит сведения о филиалах сети. Таблица содержит id_branch, branch_name, address и phone. Первичным ключом служит id_branch. Зависимость имеет вид id_branch -> branch_name, address, phone. Детерминантой признается первичный ключ. Условие НФБК выполняется.

Отношение Employees описывает работающий персонал. Таблица включает id_employee, id_user, id_branch, last_name, first_name, middle_name, phone и position. Потенциальными ключами выступают id_employee и id_user. Зависимости однозначно определяют все атрибуты через первичный ключ id_employee либо учетную запись id_user. Детерминантами служат потенциальные ключи. НФБК соблюдается.

Отношение Clients предназначено для учета посетителей. Таблица содержит id_client, id_user, last_name, first_name, middle_name и phone. Первичным ключом считается id_client, а альтернативным ключом выступает уникальный номер телефона phone. Зависимости имеют в левой части потенциальные ключи. Требования НФБК строго соблюдены.

Отношение Haircuts хранит каталог причесок. Полями таблицы выступают id_haircut, haircut_name и gender. Первичным ключом служит id_haircut. Зависимость id_haircut -> haircut_name, gender имеет детерминантой первичный ключ. НФБК выполняется.

Отношение Price_History фиксирует динамику тарифов на услуги. Таблица включает id_price_history, id_haircut, price и change_date. Первичным ключом выступает id_price_history, а составным альтернативным ключом считается пара

{id_haircut, change_date}. Детерминанты признаются суперключами. Отношение находится в НФБК.

Отношение Documents регистрирует завершенные стрижки. Таблица содержит id_document, id_branch, id_employee, id_client, id_haircut, service_date, discount_rate и final_cost. Первичным ключом выступает id_document. Все неключевые поля функционально зависят от первичного ключа. Отношение соответствует нормальной форме Бойса-Кодда.

2.3 Архитектура приложения и реализация

Архитектура программы построена на базе классического шаблона Model-View-Controller. Такое разделение позволяет отделить бизнес-логику и работу с базой данных от графического интерфейса пользователя.

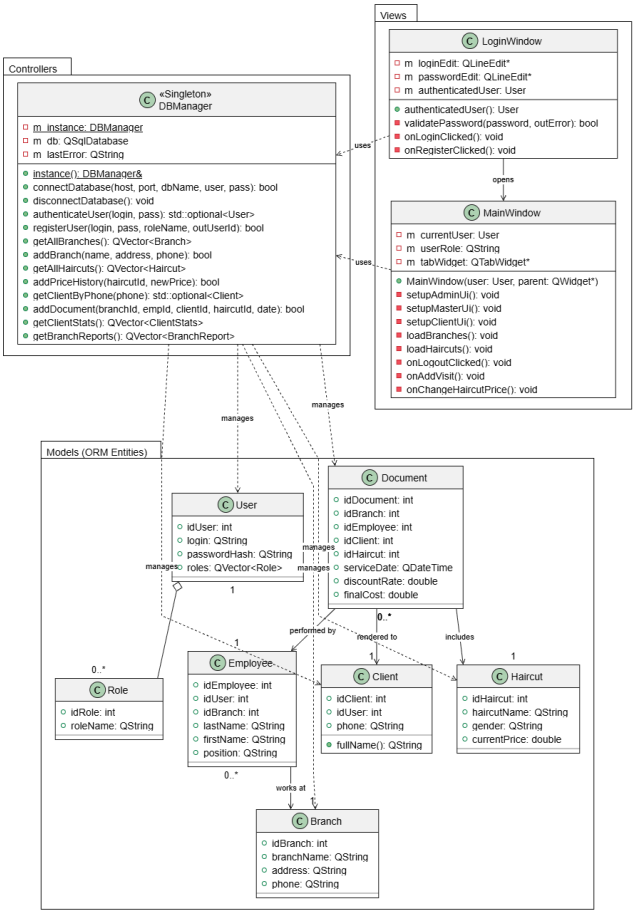


Рисунок 4 – Архитектурная диаграмма классов MVC

Уровень моделей представлен C++ структурами данных в заголовке models.h. Каждая структура строго отражает соответствующую таблицу реляционной базы данных. Созданы структуры Role, User, Branch, Employee, Client, Haircut, PriceHistory и Document.

Уровень контроллера реализован классом DBManager. Класс спроектирован по порождающему шаблону Singleton. Единственный экземпляр менеджера создается через статический метод instance. Класс инкапсулирует параметры подключения к PostgreSQL через драйвер QPSQL. Все операции выборки и обновления данных оформлены в виде методов контроллера.

Уровень представлений построен на базе библиотеки Qt Widgets. Стартовой точкой служит диалог LoginWindow. Он осуществляет проверку учетных записей и регистрацию клиентов. После авторизации создается окно MainWindow. Главное окно динамически настраивает вкладки интерфейса в соответствии с правами пользователя.

В программе реализован механизм смены пользователя. При нажатии кнопки выхода главное окно закрывается, и управление возвращается к диалогу авторизации без перезапуска приложения.

2.4 Серверная логика и безопасность данных

Часть бизнес-правил вынесена на сторону сервера PostgreSQL. Это обеспечивает независимость расчетов от клиентского кода и гарантирует целостность информации.

Для расчета стоимости стрижки и начисления накопительной скидки создан триггер trg_calculate_haircut_cost. Триггер срабатывает перед вставкой новой записи в таблицу documents. Функция fn_process_haircut_document находит актуальную цену выбранной стрижки в таблице price_history на момент визита. Затем функция подсчитывает количество предыдущих визитов клиента. Если клиент посетил салон пять или более раз, триггер автоматически выставляет

скидку 3%. Итоговая стоимость рассчитывается с учетом скидки и округляется до двух знаков после запятой.

Для ускорения аналитики созданы два представления. Представление view_client_stats объединяет данные о клиентах, числе визитов, статусе скидки и сумме расходов. Представление view_branch_report группирует визиты по подразделениям и считает суммарную выручку каждого филиала.

Безопасность системы обеспечивается комплексно. Клиентская часть проверяет сложность паролей через класс QRegularExpression. Пароль обязан содержать минимум 8 символов, цифру, заглавную букву и специальный знак. На стороне сервера на таблицу users наложено проверочное ограничение chk_password_strength. Для защиты от SQL-инъекций все обращения к базе используют исключительно параметрические запросы с методами prepare и bindValue.

3 ОСНОВНЫЕ СЦЕНАРИИ РАБОТЫ

3.1 Авторизация и разграничение прав доступа

Работа с программным комплексом начинается с окна авторизации. Пользователь вводит логин и пароль в соответствующие поля ввода.

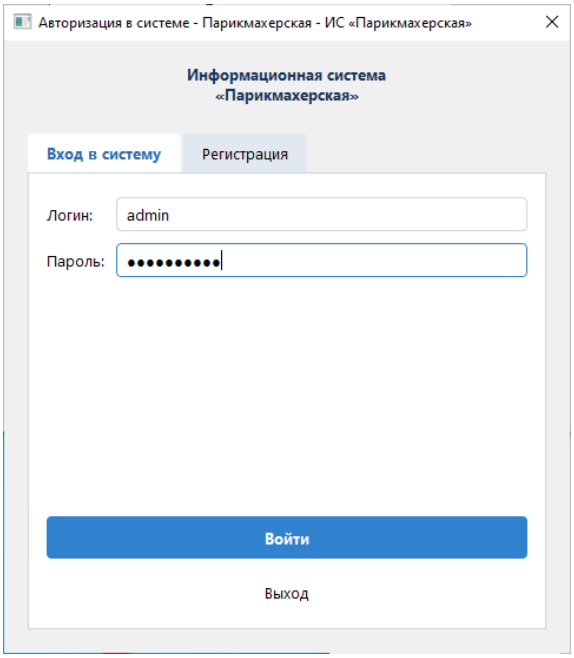


Рисунок 5 – Окно авторизации пользователя в системе

При нажатии кнопки входа контроллер DBManager проверяет корректность введенных сведений. Если пользователь не найден или пароль не совпадает, программа выводит предупреждение. Новые клиенты могут перейти на вкладку регистрации и создать аккаунт. После входа система определяет роль и формирует главное окно с доступным набором вкладок.

3.2 Сценарии работы администратора

Администратор имеет наивысшие полномочия в системе. Ему доступны все вкладки управления справочниками сети.

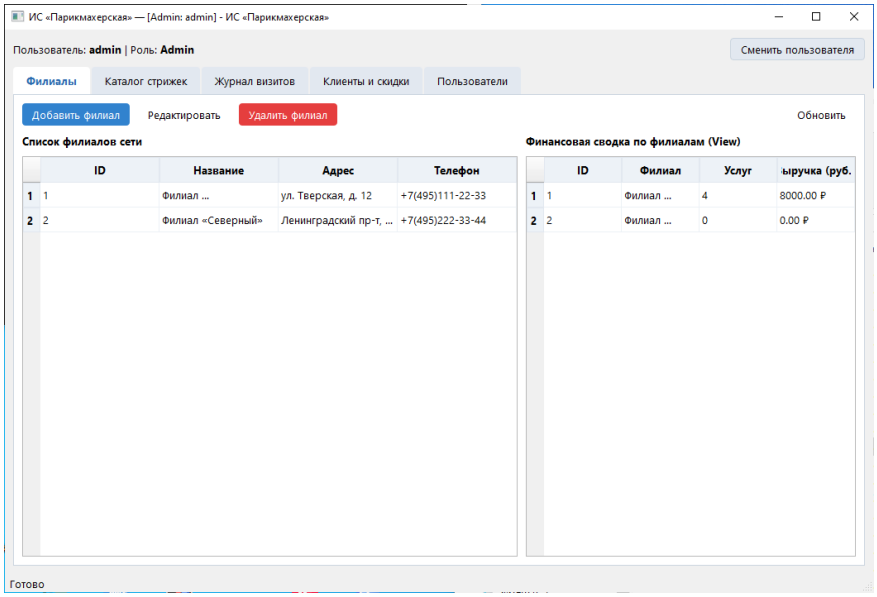


Рисунок 6 – Вкладка управления филиалами парикмахерской

На вкладке филиалов администратор просматривает список отделений, добавляет новые филиалы и корректирует адреса и телефоны. Также на вкладке отображается суммарная выручка каждого филиала.

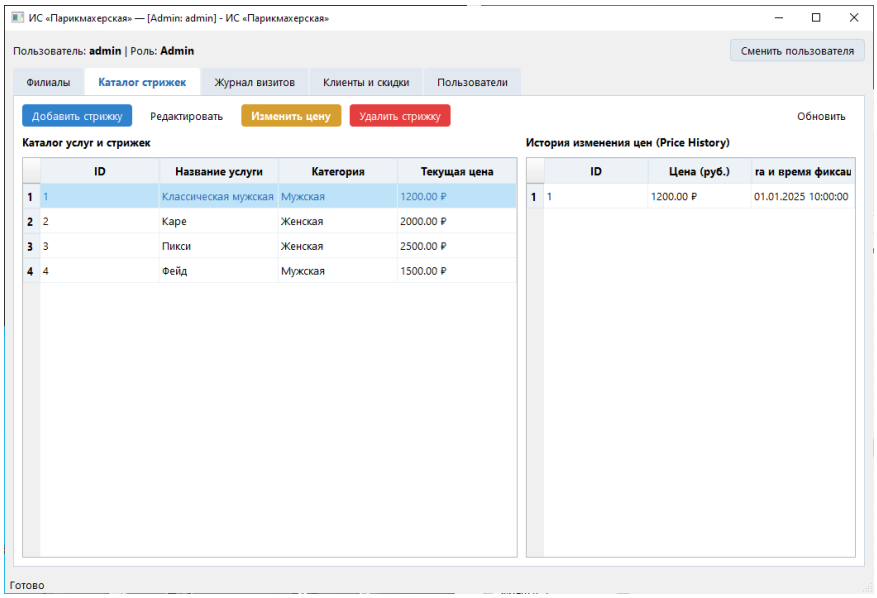


Рисунок 7 – Вкладка управления каталогом стрижек и ценами

На вкладке каталога стрижек администратор добавляет новые виды причесок и меняет тарифы. При корректировке стоимости программа

автоматически создает новую запись в таблице price_history с текущей временной меткой. Это позволяет сохранить непрерывную историю изменения тарифов.

3.3 Сценарии работы мастера

Рабочее место мастера ориентировано на фиксацию оказанных услуг. Мастер просматривает список визитов своего отделения.

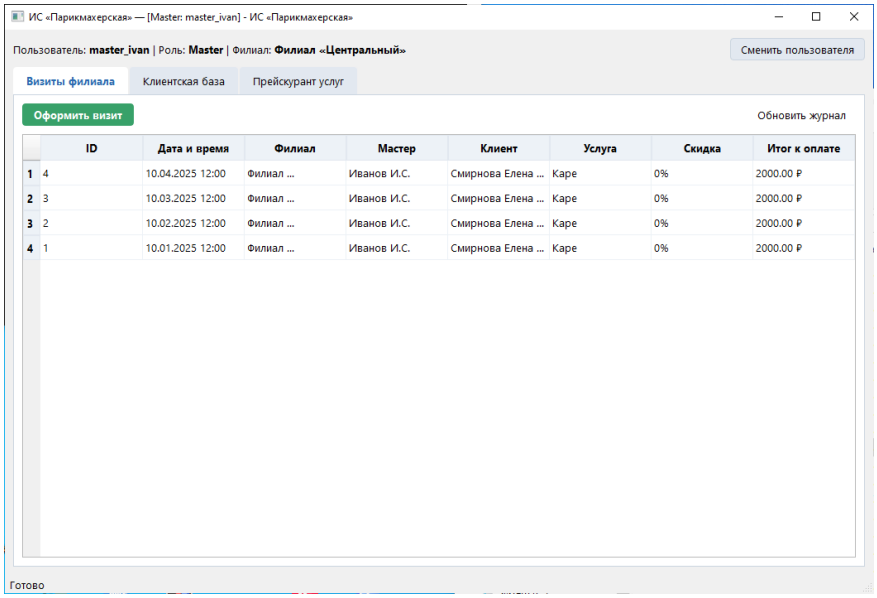


Рисунок 8 – Журнал визитов филиала в интерфейсе мастера

В журнале отображаются дата и время визита, имя клиента, выбранная стрижка, мастер, ставка скидки и итоговая цена. Кнопки удаления данных для мастера заблокированы.

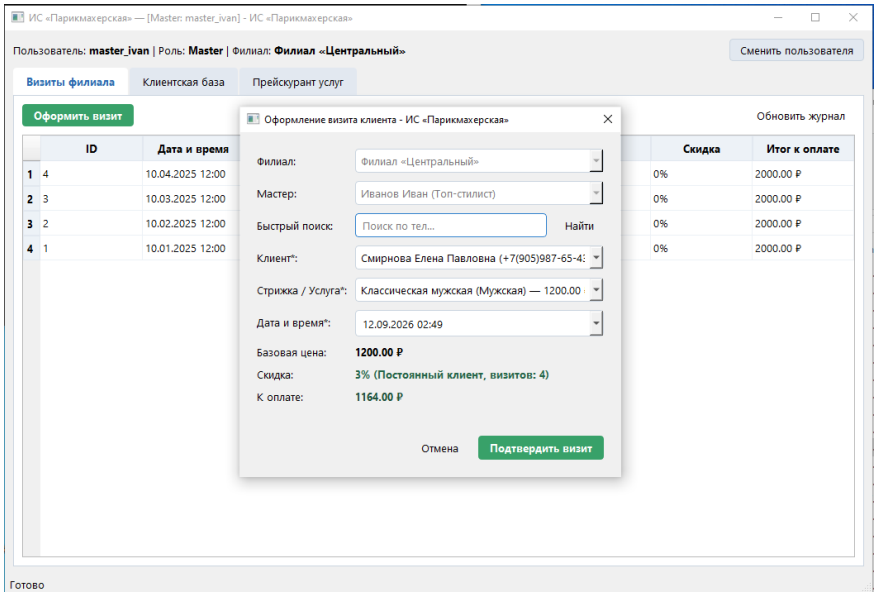


Рисунок 9 – Форма оформления нового визита клиента

Для оформления визита мастер вводит телефон клиента. Система находит человека и выводит его статус лояльности. Затем мастер выбирает стрижку из каталога. Базовая стоимость и предварительная сумма рассчитываются автоматически. При сохранении запись попадает в базу данных, и триггер фиксирует визит.

3.4 Сценарии работы клиента

Клиент имеет доступ только к собственному личному кабинету. Это исключает утечку персональных данных других посетителей.

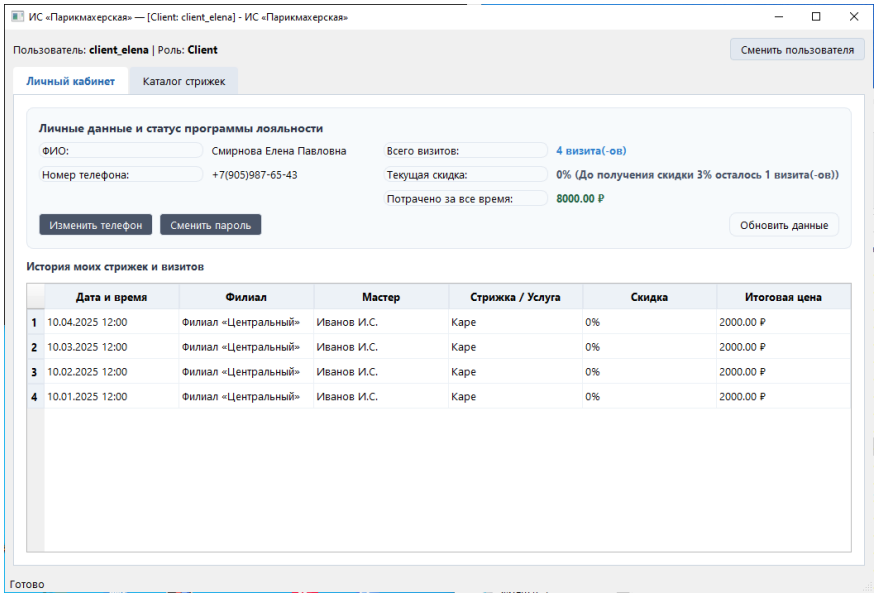


Рисунок 10 – Личный кабинет клиента со статусом скидки

В личном кабинете отображаются фамилия, имя, номер телефона, текущий размер накопительной скидки и общее количество совершенных визитов. В нижней таблице представлена история полученных услуг с точными датами и ценами. Клиент может изменить номер телефона и пароль своей учетной записи.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработана и протестирована полнофункциональная информационная система для сети парикмахерских.

В процессе работы исследована предметная область оказания парикмахерских услуг. Выделены ключевые сущности, информационные потоки и правила предоставления накопительных скидок.

Спроектирована инфологическая модель базы данных Чена. На ее основе построена реляционная схема из девяти таблиц. Для всех отношений строго доказано соответствие нормальной форме Бойса-Кодда, что исключает избыточность данных и предотвращает аномалии.

На стороне СУБД PostgreSQL реализована серверная бизнес-логика. Разработан триггер автоматического применения трехпроцентной накопительной скидки после пяти визитов. Созданы аналитические представления для оперативного формирования отчетов по филиалам и клиентам.

Клиентское приложение реализовано на C++ с библиотекой Qt 6. Архитектура выстроена по шаблону Model-View-Controller с синглтоном DBManager. В интерфейсе реализовано ролевое разделение экранов для администратора, мастера и клиента. Обеспечена безопасность данных за счет валидации паролей и параметрических запросов.

Приложение успешно собрано в виде автономного исполняемого пакета со всеми требуемыми библиотеками. Проведено успешное тестирование сценариев работы всех групп пользователей.

Полный исходный код проекта, скрипты базы данных и конфигурационные файлы размещены в открытом репозитории по адресу https://github.com/Malzan08/hairdresser-is.git.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дейт К. Дж. Введение в системы баз данных. – 8-е изд. – М., Вильямс, 2018. – 1328 с.

2. Коннолли Т., Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика. – 3-е изд. – М., Вильямс, 2017. – 1440 с.

3. Грабер М. SQL для простых смертных. – М., Лори, 2014. – 374 с.

4. Кузнецов С. Д. Основы баз данных. – 2-е изд. – М., БИНОМ. Лаборатория знаний, 2015. – 488 с.

5. Карпова Т. С. Базы данных. Модели, разработка, реализация. – СПб., Питер, 2016. – 304 с.

6. Шкрыль А. А. Разработка баз данных и приложений в среде Qt. – СПб., БХВ-Петербург, 2019. – 464 с.

7. Документация по СУБД PostgreSQL 18. – URL https://www.postgresql.org/docs/ (дата обращения 01.09.2026).

8. Документация фреймворка Qt 6. – URL https://doc.qt.io/qt-6/ (дата обращения 02.09.2026).

9. Страуструп Б. Программирование. Принципы и практика с использованием C++. – 2-е изд. – М., Вильямс, 2016. – 1328 с.

10. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб., Питер, 2020. – 368 с.

11. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. – М., Стандартинформ, 2018. – 28 с.

12. Репозиторий исходного кода курсового проекта. – URL https://github.com/Malzan08/hairdresser-is.git (дата обращения 06.09.2026).

ПРИЛОЖЕНИЕ А

Листинг модуля базы данных DBManager

```
// --- Файл src/controllers/dbmanager.h (основные объявления) ---
#ifndef DBMANAGER_H
#define DBMANAGER_H


#include <QObject>
#include <QSqlDatabase>
#include <QSqlQuery>
#include <QSqlError>
#include <QString>
#include <QVector>
#include <QVariant>
#include <optional>


#include "../models/Role.h"
#include "../models/User.h"
#include "../models/Branch.h"
#include "../models/Employee.h"
#include "../models/Client.h"
#include "../models/Haircut.h"
#include "../models/PriceHistory.h"
#include "../models/Document.h"
#include "../models/ReportStats.h"


class DBManager : public QObject {
    Q_OBJECT


public:
    static DBManager& instance();


// --- Файл src/controllers/dbmanager.cpp (подключение и синглтон) ---
#include "DBManager.h"


DBManager& DBManager::instance() {
    static DBManager instance;
    return instance;
}


DBManager::DBManager(QObject *parent) : QObject(parent) {
    if (QSqlDatabase::contains("hairdresser_connection")) {
        m_db = QSqlDatabase::database("hairdresser_connection");
    } else {
        m_db = QSqlDatabase::addDatabase("QPSQL", "hairdresser_connection");
    }
}


DBManager::~DBManager() {
    disconnectDatabase();
}


bool DBManager::connectDatabase(const QString &host, int port, const QString &dbName, const QString &user, const
QString &password) {
    if (m_db.isOpen()) {
        m_db.close();
    }
    m_db.setHostName(host);
    m_db.setPort(port);
```

ПРИЛОЖЕНИЕ Б

Триггер и аналитические представления базы данных

```
-- Триггерная функция и триггер расчета стоимости услуги
CREATE OR REPLACE FUNCTION fn_process_haircut_document()
RETURNS TRIGGER AS $$
DECLARE
    v_base_price NUMERIC(10, 2);
    v_visit_count INT;
BEGIN
    SELECT price INTO v_base_price
    FROM price_history
    WHERE id_haircut = NEW.id_haircut AND change_date <= NEW.service_date
    ORDER BY change_date DESC
    LIMIT 1;


    IF v_base_price IS NULL THEN
        SELECT price INTO v_base_price FROM price_history
        WHERE id_haircut = NEW.id_haircut ORDER BY change_date DESC LIMIT 1;
    END IF;


    IF v_base_price IS NULL THEN
        RAISE EXCEPTION 'Для выбранной стрижки не установлена стоимость в истории цен!';
    END IF;


    SELECT COUNT(*) INTO v_visit_count
    FROM documents
    WHERE id_client = NEW.id_client;


    IF v_visit_count >= 4 THEN
        NEW.discount_rate := 3.00;
    ELSE
        NEW.discount_rate := 0.00;
    END IF;


    NEW.final_cost := ROUND(v_base_price * (1.0 - (NEW.discount_rate / 100.0)), 2);


    RETURN NEW;
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_calculate_haircut_cost
BEFORE INSERT ON documents
FOR EACH ROW
EXECUTE FUNCTION fn_process_haircut_document();
```

ПРИЛОЖЕНИЕ В

Скрипты разграничения прав доступа (DCL)

```
-- Роль Администратора (полные права на управление схемой и данными)
CREATE ROLE admin_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_role;


-- Роль Мастера (чтение справочников, регистрация клиентов и визитов)
CREATE ROLE master_role;
GRANT SELECT ON branches, haircuts, price_history, employees TO master_role;
GRANT SELECT, INSERT, UPDATE ON clients TO master_role;
GRANT SELECT, INSERT ON documents TO master_role;
GRANT USAGE, SELECT ON SEQUENCE documents_id_document_seq, clients_id_client_seq TO master_role;
GRANT SELECT ON view_branch_report, view_client_stats TO master_role;


-- Роль Клиента (чтение каталога услуг, доступ к личной статистике)
CREATE ROLE client_role;
GRANT SELECT ON haircuts, price_history TO client_role;
GRANT SELECT ON view_client_stats TO client_role;
GRANT SELECT, UPDATE(phone) ON clients TO client_role;
GRANT SELECT, UPDATE(password_hash) ON users TO client_role;
```